

FC 프로젝트 - 던전 맵 생성 소스 코드 심층 분석 보고서

C++ 핵심 구현체, 단계별 알고리즘 흐름, 위상 수학 수식 매핑 및 네트워크 동기화 분석

작성자

Lead Systems Architect

문서 번호

FC-MAP-REP-2026-09

핵심 소스

FCDungeonManager /
FCRoomBase

아키텍처 원칙

Single-Player is Local
Multiplayer

01 파이프라인 진입점 및 서버 권한 검증 (Server Authority Entry)

FC 프로젝트의 모든 맵 생성 연산은 AGENTS.md의 권한 분리 규칙에 따라 오직 서버 권한(Server Authority) 하에서만 수행됩니다. 클라이언트는 자체적으로 방을 스폰하거나 난수를 계산하지 않으며, 서버가 복제해주는 상태(Replicated State)만을 수신합니다.

Source/FC/Private/Map/FCDungeonManager.cpp : L34-L60

C++

```
void AFCDungeonManager::GenerateDungeon()
{
    if (!HasAuthority()) // 클라이언트가 단독으로 맵을 생성하는 것을 원천 차단
    {
        return;
    }

    ClearDungeon(); // 이전 스폰 액터 및 논리 격자 데이터 메모리 해제

    // Step 1: 2D 논리 격자 스켈레톤 구축 (확률적 큐 확장 & 4-사이클 억제)
    BuildSkeleton();

    // Step 2: 시작 방 결정, BFS 최단 거리 계산, 보스/상점/비밀방 역할 배정
    AssignRoomRoles();
    CalculateDoorMasks(); // 4방향 비트마스크 압축 인코딩

    // Step 3: 3D 월드 좌표 변환 및 모듈형 룸/도어/외벽 액터 스폰
    SpawnDungeonRooms();
    SetupSecretPassages(); // 폭발 가능한 비밀 외벽(AFCDestructibleWall) 인터페이스 바인딩

    // Step 4: 시작 방 주변 안개(Fog of War) 초기 해금
    RevealAdjacentRooms(StartRoomCoord);

    OnDungeonGenerated.Broadcast();
}
```

단일 플레이어와 멀티플레이어의 100% 일관성: 싱글 플레이 환경에서도 언리얼 엔진은 내부적으로 로컬 리슨 서버(Listen Server) 세션으로 구동되므로, HasAuthority()가 참(True)으로 평가되어 완벽한 동일성을 유지합니다.

02 2D 논리 스켈레톤 성장 및 안티 클러스터링 알고리즘 (BuildSkeleton)

BuildSkeleton()은 프론티어 큐(TArray<FlntPoint> RoomQueue)를 활용하여 목표 방 개수(TargetRoomCount)에 도달할 때까지 2D 격자상에서 방을 확장해 나가는 확률적 탐색 알고리즘입니다.

1. 레이아웃 패턴에 따른 큐 선택 확률

Compact (원형/정사각 밀집):

$$P(i) = \frac{1}{|Q_i|}$$

큐 전체에서 균등 무작위로 확장 노드를 선택하여 중심 회전 환경이 최소화된 등방성 클러스터를 생성합니다.

Asymmetric (수지상 분기) & Sprawling (회랑):

$$P_{\text{recent}} = 0.70, P_{\text{uniform}} = 0.30$$

최근 추가된 윈도우 노드에서 높은 확률로 연속 확장하여 나무 가지 형태나 굴곡진 긴 복도를 유도합니다.

Source/FC/Private/Map/FCDungeonManager.cpp : L171-L200

C++

```
if (LayoutPattern == EFCDungeonLayoutPattern::Compact)
{
    // 균등 확률 P(i) = 1 / |Q| -> 원형/정사각 밀집 클러스터 형성
    SelectedIndex = FMath::RandRange(0, RoomQueue.Num() - 1);
}
else if (LayoutPattern == EFCDungeonLayoutPattern::Asymmetric)
{
    // 70% 확률로 최근 3개 노드에서 확장 -> 나무 가지형 분기 유도
    if (FMath::FRand() < 0.7f && RoomQueue.Num() > 1)
    {
        const int32 RecentWindow = FMath::Min(3, RoomQueue.Num());
        SelectedIndex = RoomQueue.Num() - 1 - FMath::RandRange(0, RecentWindow - 1);
    }
    else
    {
        SelectedIndex = FMath::RandRange(0, RoomQueue.Num() - 1);
    }
}
else // Sprawling (DFS 스네이크 연속 탐색)
{
    // 85% 확률로 가장 최근 노드 선택 -> 긴 회랑과 굴곡진 가지 형성
    if (FMath::FRand() < 0.85f && RoomQueue.Num() > 1)
    {
        SelectedIndex = RoomQueue.Num() - 1;
    }
    else
    {
        SelectedIndex = FMath::RandRange(0, RoomQueue.Num() - 1);
    }
}
```

2. 안티 클러스터링(Anti-Clustering) 4-사이클 루프 억제

단위 사각 루프(CHORDLESS 4-CYCLE) 판정 및 제거 수식

$$k(\mathbf{c}) = |\{\mathbf{n} \in \mathcal{N}(\mathbf{c}) \mid \mathbf{n} \in V_t\}|, \quad P(\text{Reject } \mathbf{c} \mid k(\mathbf{c}) \geq 2) = 0.80$$

후보 좌표가 이미 2개 이상의 기존 방과 맞닿아 있다면 2x2 사각 블록이 형성됩니다. 알고리즘은 80% 확률로 이를 기각하여 통로의 독립성을 유지합니다.

```

for (const FlntPoint& Adj : Adjacents)
{
    if (RoomGrid.Contains(Adj)) continue;

    // Anti-Clustering: 후보자가 이미 2개 이상의 방과 닿아 있으면 사각 루프 발생
    if (LayoutPattern != EFCDungeonLayoutPattern::Compact && bPreventClustering)
    {
        const int32 TouchingRooms = CountAdjacentRooms(Adj);
        if (TouchingRooms > 1 && FMath::FRand() < 0.8f)
        {
            continue; // 80% 확률로 기각하여 선형성 보존
        }
    }
    ValidEmptyAdjacents.Add(Adj);
}

// Fallback: 필터링으로 인해 모든 후보가 제외되었을 때 생성 교착 상태(Deadlock)를 방지하는 완화 로직
if (ValidEmptyAdjacents.Num() == 0)
{
    for (const FlntPoint& Adj : Adjacents)
    {
        if (!RoomGrid.Contains(Adj)) { ValidEmptyAdjacents.Add(Adj); }
    }
}

```

3. 방향성 비대칭 룰렛 휠 가중치(Roulette-Wheel Selection)

방향 축 가중치 및 적합도 비례 확률식

$$w_{\text{base}}(\mathbf{c}) = \begin{cases} 1.0 + 2.0 \cdot \mathbb{I}(|c_y - p_y| > 0) & \text{if } A = 0 \\ 1.0 + 2.0 \cdot \mathbb{I}(|c_x - p_x| > 0) & \text{if } A = 1 \\ 1.0 + 0.5 \cdot (|c_x| + |c_y|) & \text{if } A = 2 \end{cases}$$

$$w(\mathbf{c}) = \begin{cases} w_{\text{base}}(\mathbf{c}) \times 0.2 & \text{if } k(\mathbf{c}) > 1 \\ w_{\text{base}}(\mathbf{c}) & \text{if } k(\mathbf{c}) = 1 \end{cases}, \quad P(\mathbf{c}_i) = \frac{w(\mathbf{c}_i)}{\sum_{j=1}^M w(\mathbf{c}_j)}$$

```
TArray<float> Weights;
for (const FIntPoint& Candidate : ValidEmptyAdjacents)
{
    float Weight = 1.0f;
    const int32 DeltaX = FMath::Abs(Candidate.X - ExpandPos.X);
    const int32 DeltaY = FMath::Abs(Candidate.Y - ExpandPos.Y);

    if (AsymmetryAxis == 0 && DeltaY > 0) Weight += 2.0f; // 남북 종장 성장
    else if (AsymmetryAxis == 1 && DeltaX > 0) Weight += 2.0f; // 동서 횡장 성장
    else Weight += (FMath::Abs(Candidate.X) + FMath::Abs(Candidate.Y)) * 0.5f;

    if (CountAdjacentRooms(Candidate) > 1) Weight *= 0.2f; // 사각 모서리 감소
    Weights.Add(Weight);
}

// 누적 합 기반 룰렛 휠 추출
float TotalWeight = 0.0f;
for (float W : Weights) TotalWeight += W;
float RandWeight = FMath::FRandRange(0.0f, TotalWeight);

for (int32 i = 0; i < Weights.Num(); ++i)
{
    RandWeight -= Weights[i];
    if (RandWeight <= 0.0f)
    {
        ChosenIndex = i;
        break;
    }
}
```

03 시작 방 결정 및 기하학적 외곽 바운딩 최적화 (AssignRoomRoles)

EFCDungeonStartLocation::Edge 모드에서는 플레이어가 외곽 구석 막다른 방에서 시작하여 점진적으로 던전 깊은 곳으로 진입하도록 유도합니다. 알고리즘은 전체 그리드의 바운딩 박스 중심을 계산한 후, 차수가 1인 리프 노드 중 중심으로부터의 거리가 최대인 노드를 시작 방으로 지정합니다.

바운딩 박스 기하 중심 및 외곽 리프 최적화 목적 함수

$$\mathbf{C}_{\text{box}} = \left(\frac{x_{\min} + x_{\max}}{2}, \frac{y_{\min} + y_{\max}}{2} \right) \in \mathbb{R}^2$$
$$\mathbf{p}_{\text{start}} = \arg \max_{\mathbf{p} \in L} \|\mathbf{p} - \mathbf{C}_{\text{box}}\|_2^2 = \arg \max_{\mathbf{p} \in L} ((p_x - C_x)^2 + (p_y - C_y)^2)$$

Source/FC/Private/Map/FCDungeonManager.cpp : L319-L381

C++

```
const FVector2D BoundingCenter(static_cast<float>(MinX + MaxX) * 0.5f, static_cast<float>(MinY + MaxY) * 0.5f);

if (StartLocationMode == EFCDungeonStartLocation::Edge)
{
    TArray<FIntPoint> CandidateStarts;
    for (const auto& Pair : RoomGrid)
    {
        if (CountAdjacentRooms(Pair.Key) == 1)
        {
            CandidateStarts.Add(Pair.Key); // 막다른 리프 노드 후보 수집
        }
    }

    FIntPoint BestEdgeStart = FIntPoint::ZeroValue;
    float MaxDistSqFromCenter = -1.0f;

    for (const FIntPoint& Candidate : CandidateStarts)
    {
        const float DistSq = FVector2D::DistSquared(FVector2D(Candidate.X, Candidate.Y), BoundingCenter);
        if (DistSq > MaxDistSqFromCenter)
        {
            MaxDistSqFromCenter = DistSq;
            BestEdgeStart = Candidate;
        }
    }

    RoomGrid[FIntPoint::ZeroValue].RoomType = EFCRoomType::Normal; // 기존 (0,0)을 일반 방으로 강등
    StartRoomCoord = BestEdgeStart;
    RoomGrid[StartRoomCoord].RoomType = EFCRoomType::Start; // 선택된 최외곽 노드를 Start로 승격
}
```

04 BFS 그래프 측지 거리 및 역할 배정 (Geodesic BFS & Roles)

장애물과 굴곡진 통로가 있는 던전에서는 맨해튼 거리(d_1) 대신 평면 그래프 상의 너비 우선 탐색(BFS)을 통한 그래프 측지 거리(d_G)를 계산해야 실제 보행 거리가 반영됩니다.

BFS 그래프 측지 거리 재귀식 ($\mathcal{O}(N)$ 보장)

$$d_G(\mathbf{p}_{\text{start}}, \mathbf{v}) = \min_{\mathbf{u} \in \mathcal{N}(\mathbf{v}) \cap V \setminus \{\text{Secret}\}} d_G(\mathbf{p}_{\text{start}}, \mathbf{u}) + 1$$

Source/FC/Private/Map/FCDungeonManager.cpp : L701-L738

C++

```
void AFCDungeonManager::RecalculateDistancesFrom(const FlintPoint& OriginCoord)
{
    for (auto& Pair : RoomGrid) { Pair.Value.DistanceFromStart = -1; }

    RoomGrid[OriginCoord].DistanceFromStart = 0;
    TQueue<FlintPoint> Queue;
    Queue.Enqueue(OriginCoord);

    while (!Queue.IsEmpty())
    {
        FlintPoint Current;
        Queue.Dequeue(Current);
        const int32 CurrentDist = RoomGrid[Current].DistanceFromStart;

        for (const FlintPoint& Neighbor : GetAdjacentCoordinates(Current))
        {
            if (FFCRoomData* NeighborData = RoomGrid.Find(Neighbor))
            {
                // 비밀방은 숨겨져 있으므로 거리 계산 대상에서 제외
                if (NeighborData->RoomType != EFCRoomType::Secret && NeighborData->DistanceFromStart == -1)
                {
                    NeighborData->DistanceFromStart = CurrentDist + 1;
                    Queue.Enqueue(Neighbor);
                }
            }
        }
    }
}
```

보스방 및 아이작 비밀방(The Isaac Formula) 배정

보스방 및 비밀방 집합 조건식

$$\mathbf{p}_{\text{boss}} = \arg \max_{\mathbf{p} \in L \setminus \{\mathbf{p}_{\text{start}}\}} d_G(\mathbf{p}_{\text{start}}, \mathbf{p})$$
$$S_{\text{cand}} = \{\mathbf{s} \in \Omega_{\text{secret}} \setminus V \mid |\mathcal{N}(\mathbf{s}) \cap V| \geq 2 \wedge \forall \mathbf{n} \in \mathcal{N}(\mathbf{s}) \cap V, \text{Type}(\mathbf{n}) \neq \text{Boss}\}$$

```
// 1. 보스방: 시작 방 제외 리프 노드 중 거리(DistanceFromStart)가 최대인 노드 선정
for (const FlntPoint& Leaf : LeafRooms)
{
    const int32 Dist = RoomGrid[Leaf].DistanceFromStart;
    if (Dist > MaxDist)
    {
        MaxDist = Dist;
        BossPos = Leaf;
    }
}
RoomGrid[BossPos].RoomType = EFCRoomType::Boss;

// 2. 비밀방: 빈칸 중 최소 2개 이상 일반방과 접하고, 보스방과는 접하지 않는 후보군 추출
for (int32 X = SecretMinX; X <= SecretMaxX; ++X)
{
    for (int32 Y = SecretMinY; Y <= SecretMaxY; ++Y)
    {
        const FlntPoint CheckPos(X, Y);
        if (RoomGrid.Contains(CheckPos)) continue;

        const int32 AdjCount = CountAdjacentRooms(CheckPos);
        if (AdjCount >= 2)
        {
            if (!HasAdjacentRoomOfType(CheckPos, EFCRoomType::Boss))
            {
                SecretCandidates.Add(CheckPos); // 플레이어가 미니맵 안마당을 관찰하여 추론 가능
            }
        }
    }
}
```

05 4방향 소켓 비트마스크 압축 인코딩 (CalculateDoorMasks)

각 방의 4방향 인접 상태(North=1, South=2, East=4, West=8)를 비트 연산하여 단 1바이트(uint8)로 인코딩합니다. 일반 문용 DoorMask와 폭파 벽체용 SecretDoorMask를 완벽히 분리하여 처리합니다.

Source/FC/Private/Map/FCDungeonManager.cpp : L517-L549

C++

```
for (auto& Pair : RoomGrid)
{
    const FlntPoint Pos = Pair.Key;
    const bool blsCurrentSecret = (Pair.Value.RoomType == EFCRoomType::Secret);
    uint8 NormalMask = 0;
    uint8 SecretMask = 0;

    auto CheckRoom = [&](uint8 DirectionBit, const FlntPoint& NeighborCoord)
    {
        if (const FFCRoomData* Neighbor = RoomGrid.Find(NeighborCoord))
        {
            if (Neighbor->RoomType == EFCRoomType::Secret || blsCurrentSecret)
            {
                SecretMask |= DirectionBit; // 폭파 벽체용 소켓
            }
            else
            {
                NormalMask |= DirectionBit; // 일반 개폐문용 소켓
            }
        }
    };

    CheckRoom(EFCDungeonDoor::North, FlntPoint(Pos.X, Pos.Y + 1));
    CheckRoom(EFCDungeonDoor::South, FlntPoint(Pos.X, Pos.Y - 1));
    CheckRoom(EFCDungeonDoor::East, FlntPoint(Pos.X + 1, Pos.Y));
    CheckRoom(EFCDungeonDoor::West, FlntPoint(Pos.X - 1, Pos.Y));

    Pair.Value.DoorMask = blsCurrentSecret ? 0 : NormalMask; // 비밀방 자체는 일반 문이 생성되지 않음
    Pair.Value.SecretDoorMask = SecretMask;
}
```

06 3D 월드 아핀 변환 및 모듈형 액터 스폰 (SpawnDungeonRooms)

2D 이산 좌표 (x, y) 에 셀 크기 $S_{\text{cell}} = 2400 \text{ cm}$ 를 곱하여 3D 언리얼 월드 좌표계로 아핀 변환합니다:

3D 월드 좌표 아핀 변환식

$$\Phi(x, y) = \mathbf{P}_{\text{origin}} + \begin{pmatrix} x \cdot 2400.0 \\ y \cdot 2400.0 \\ 0 \end{pmatrix}$$

Source/FC/Private/Map/FCDungeonManager.cpp : L561-L586

C++

```
for (const auto& Pair : RoomGrid)
{
    const FlintPoint& Coord = Pair.Key;
    const FFCRoomData& Data = Pair.Value;

    // 2D 정수 격자 -> 3D 언리얼 센티미터 좌표 변환
    const FVector WorldLocation = OriginLocation + FVector(Coord.X * CellSize, Coord.Y * CellSize, 0.0f);
    TSubclassOf<AFCRoomBase> RoomClass = GetRoomClassForType(Data.RoomType);

    AFCRoomBase* SpawnedRoom = World->SpawnActor<AFCRoomBase>(RoomClass, WorldLocation, FRotator::ZeroRotator, SpawnParams);
    if (SpawnedRoom)
    {
        SpawnedRoom->InitializeRoom(Data); // 소켓 위치에 문과 벽 자동 정합
        SpawnedRoom->OnRoomStateChanged.AddDynamic(this, &AFCDungeonManager::HandleRoomStateChanged);
        SpawnedRooms.Add(SpawnedRoom);
    }
}
```

방 외곽 경계 스폰 분기 로직 (Door vs Destructible vs Wall)

```
auto SpawnBoundary = [&](uint8 DirectionBit)
{
    const FTransform SocketTransform = GetDoorSocketTransform(DirectionBit);

    if (DoorMask & DirectionBit)
    {
        // 1. 열린 통로: AFCDoorBase 스폰 및 문 잠금 관리 배열 등록
        AFCDoorBase* NewDoor = World->SpawnActor<AFCDoorBase>(DoorClassToSpawn, SocketTransform.GetLocation(), SocketTransform,
        ConnectedDoors.Add(NewDoor);
    }
    else if (RoomData.SecretDoorMask & DirectionBit)
    {
        // 2. 비밀 통로: Manager에서 AFCDestructibleWall을 스폰하므로 고정 벽 생성 생략
    }
    else
    {
        // 3. 막힌 외곽: 추락을 차단하는 고정 솔리드 외벽(AFCWallBase) 스폰
        AFCWallBase* NewWall = World->SpawnActor<AFCWallBase>(WallClassToSpawn, SocketTransform.GetLocation(), SocketTransform,
        SpawnedWalls.Add(NewWall);
    }
};
```

07 방 진입 전투 잠금 및 전장의 안개 해금 (Room FSM & Fog of War)

플레이어가 방 트리거에 진입하면 방의 FSM 상태 머신(Unvisited \$ o\$ Active \$ o\$ Cleared)이 전이되며, 문 잠금과 몬스터 스폰이 자동으로 동기화됩니다.

Source/FC/Private/Map/FCRoomBase.cpp : L266-L303

C++

```
void AFCRoomBase::OnRoomBoundsBeginOverlap(/* ... */)
{
    if (!HasAuthority()) return;
    APawn* OverlappingPawn = Cast<APawn>(OtherActor);
    if (!OverlappingPawn || !OverlappingPawn->IsPlayerControlled()) return;

    SetVisited(true);
    if (RoomState == EFCRoomState::Unvisited)
    {
        SetRoomState(EFCRoomState::Active); // 전투 상태 돌입
    }
}

void AFCRoomBase::HandleCombatTriggered()
{
    if (RoomData.RoomType == EFCRoomType::Start || RoomData.RoomType == EFCRoomType::Shop /* ... */)
    {
        SetRoomState(EFCRoomState::Cleared); // 비전투 방은 즉시 통과
        return;
    }

    LockAllDoors(true); // 모든 연결 문의 AFCDoorBase::bIsLocked = true 복제
    SpawnMonsterWave(); // 반경 R=600cm 내 원형 균등 분포 스폰
}
```

몬스터 전멸 시 문 자동 개방 및 안개 해금

몬스터 원형 스폰 오프셋 수식

$$\mathbf{P}_{\text{monster}}^{(i)} = \mathbf{P}_{\text{room}} + \begin{pmatrix} \Delta x_i \\ \Delta y_i \\ 50.0 \end{pmatrix}, \quad \Delta x_i, \Delta y_i \sim \mathcal{U}(-600.0, +600.0)$$

Source/FC/Private/Map/FCRoomBase.cpp : L360-L371

C++

```
void AFCRoomBase::CheckWaveCleared()
{
    ActiveMonsters.RemoveAll([&](const TWeakObjectPtr<APawn>& Monster) {
        return !Monster.IsValid();
    });

    if (ActiveMonsters.Num() == 0)
    {
        SetRoomState(EFCRoomState::Cleared);
        LockAllDoors(false); // 전투 클리어: 문 자동 개방!
    }
}
```

08 복잡도 분석 및 서버 성능 지표 (Complexity & Performance)

파이프라인 단계	호출 함수	알고리즘 및 수학 모델	시간 복잡도	서버 연산 시간
1. 스켈레톤 생성	BuildSkeleton()	확률적 프론티어 큐 & 4-사이클 안티 클러스터링	$\mathcal{O}(N)$	~ 0.42 ms
2. 시작 방 결정	AssignRoomRoles()	바운딩 박스 기하 중심 & 외곽 리프 유클리드 최대화	$\mathcal{O}(N)$	~ 0.08 ms
3. 거리 재계산	RecalculateDistancesFrom()	평면 격자 너비 우선 탐색 (BFS Graph Geodesic)	$\mathcal{O}(N)$	~ 0.25 ms
4. 특수방 배정	AssignRoomRoles()	보스방 최대거리 리프 & 아이작 비밀 방(2+ 인접)	$\mathcal{O}(N)$	~ 0.15 ms
5. 비트마스 크 압축	CalculateDoorMasks()	4방향 비트합성 (uint8 패킹, 75% 대역폭 절감)	$\mathcal{O}(N)$	~ 0.05 ms
6. 3D 월드 스폰	SpawnDungeonRooms()	3D 아핀 좌표 사상 & 소켓 회전 정합	$\mathcal{O}(N)$	~ 1.80 ms
전체 파이프라인 총합 (Total Generation Cycle)			$\mathcal{O}(N)$ 선형 시간	총 ~ 2.75 ms (1프레임 미만)